
ISyE 6740 - Fall 2025

Final Project

Team Member Names: Zachary Allen (Group 212)

Project Title: Evaluating Entity Resolution Methods on Synthetic Patient Records

Problem Statement

Entity resolution, also known as record linkage or deduplication, is a critical challenge in healthcare data management as patient records are frequently duplicated across and within healthcare systems. For instance, a patient who visits a hospital multiple times over many years may have multiple records in a database corresponding to each visit. It is common and reasonable for these records to contain changes in key identifiers such as address, name, marital status, etc. or typos due to data entry errors. In an ideal case, a unique identifier would be used to track individuals whose information appears multiple times, however, in practice implementing such an identifier can be difficult.

As an example, the Center for Medicare and Medicaid Services (CMS) maintains the Transformed Medicaid Statistical Information System (T-MSIS), a comprehensive national database to collect detailed data on beneficiaries, enrollment services, and payments for the Medicaid and Children's Health Insurance Program (CHIP) from all U.S. states and territories. In the T-MSIS, MSIS identification numbers (and/or SSNs) are used to represent individuals with distinct demographic and eligibility information over time [1]. A major challenge CMS has identified with making the MSIS IDs consistent is that different states have different procedures for assigning them, especially for newborns whose mothers are eligible for Medicaid or pregnancy-related benefits. This can result in the same individual appearing in a dataset multiple times without a consistent identifier between records.

This project attempts to evaluate multiple entity resolution approaches on synthetic patient data in an effort to tackle challenges similar to what CMS is facing. Each approach will be evaluated on their ability to identify duplicate patient records using a series of quantitative metrics such as precision, recall, and f1-score.

Data Source

This project utilizes the SyntheticMass dataset (2017 version), a comprehensive collection of synthetic patient records developed by the MITRE Corporation as part of the Synthea project [2]. SyntheticMass was specifically designed to simulate realistic patient populations while containing no actual Personally Identifiable Information (PII).

The SyntheticMass 2017 dataset contains over one million unique synthetic patient records from Massachusetts. Each record includes both demographic and administrative information that can be utilized for entity resolution tasks, the ones used for this project are listed below:

- Names: First name and Last name
- Demographic: Date of Birth and Gender
- Geographic: Street Address, City, State, and ZIP Code
- Admin: ID (unique string)

For quicker analysis, a subset of 50,000 records were used from the SyntheticMass dataset. Since each record is unique, 80% of records were duplicated 1 to 5 times (chosen randomly) with each duplication being partially altered to replicate data quality issues such as typos, address changes, and missing values. Appendix A.1 contains the full procedure for duplication which resulted in a total of 169,628 records. The table below shows an example where the first record (original) was duplicated 3 additional times with partial changes to key fields (shown in bold). Note that in the resulting dataset records with the same ID value correspond to the same individual. The objective of the models will be to cluster records that correspond to the same individual regardless of data quality issues.

ID	FIRST_C	LAST_C	ADDRESS_LINE_1	ADDRESS_CITY	BIRTHDATE
40632	Lonzo	Lockman	219 ELVERA RDG	SOUTH HADLEY	1955-03-20
40632	Lonoz	None	4836 Nicholson Cape	None	1955-03-20
40632	Lnozo	Bowman	219 ELVERA RDG	SOUTH HADLEY	1955-04-20
40632	Lonoz	Lockman	219 ELVERA RDG	SOUTH HbADLEY	1955-03-20

Methodology

This project will implement and evaluate the following three entity resolution approaches of increasing complexity. However, to reduce the overall number of comparisons between records, all three methods will incorporate blocking.

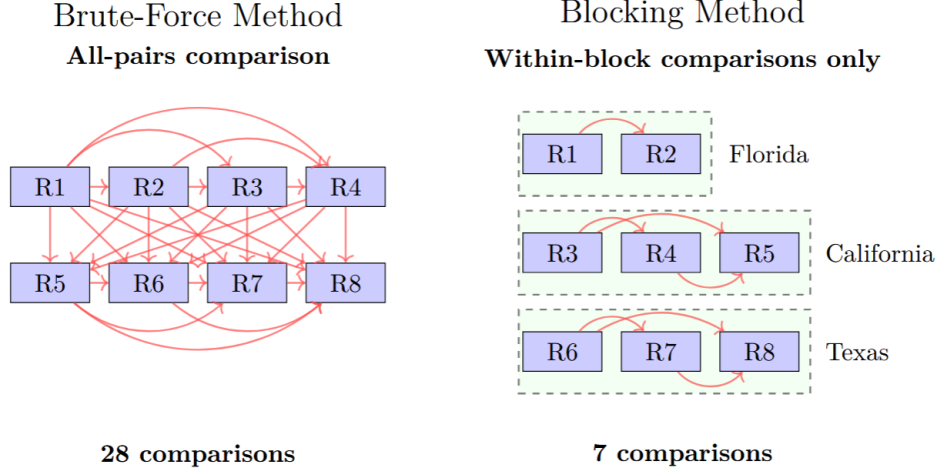
Blocking

In a *brute-force entity resolution methodology*, one would compare a given record in a dataset to every other record with calculations to determine whether the two records represent the same entity. However, the number of comparisons required with this approach scales quadratically ($O(n^2)$) with the size of the dataset, causing this method to be computationally infeasible for large datasets.

A common approach to improve the brute-force method is by using a *blocking method*, where, rather than comparing each record to every other record, each record is placed into a block (group) based on one or more shared fields. Then, for each block, compare each record to every other record within the same block. Since a record can only be within one block, this significantly reduces the total number of comparisons required.

As an example, consider a dataset containing eight records, R1 through R8. In the brute-force method, comparing each record to every other record results in $\binom{8}{2} = 28$ comparisons. In the

blocking method, R1 through R8 are grouped depending on which state they belong to, then only records within the same state are compared against each other. This results in $\binom{2}{2} + \binom{3}{2} + \binom{3}{2} = 7$ total comparisons.



In general, it can be shown that a blocking method with G groups, each of size g_i , compared to a brute-force method with N records such that $\sum_i g_i = N$ will be

$$\sum_{i=1}^G \binom{g_i}{2} \leq \binom{N}{2}$$

The key disadvantage with the blocking method is that if a record is misplaced into the incorrect block due to a typo or other error with the blocking fields, it will not be compared to records in its intended, actual block. For the purposes of this project, the **BIRTHPLACE** and **GENDER** fields are assumed to be accurate across the dataset and will be used for blocking purposes.

1. Simple String Comparison

The baseline approach for entity resolution will be a simple string comparison. That is, all target fields for each record (first name, last name, gender, street address, etc.) will be concatenated into a single string. Then, each record string within the same block will be compared against one another using the Normalized Indel Similarity [3]. Records whose similarity scores are above a certain threshold will be considered the same individual and assigned the same entity identifier (Cluster ID).

$$\text{Normalized Indel Similarity}(s_1, s_2) = 1 - \frac{\text{Levenshtein Distance}}{\max(\text{len}(s_1), \text{len}(s_2))}$$

2. Custom Similarity Formula with Agglomerative Clustering

This approach computes pairwise similarity scores between records using a weighted combination of field-specific similarity measures, then applies hierarchal clustering on the distance matrix for each block. The table below shows each of the fields, metrics, and weights used in the similarity formula. The result of comparing two records within the same block will be a weighted score between 0 and 1 (inclusive) representing the similarity. The process of creating the score will inherently incorporate more information compared to the Simple String Comparison approach.

After comparing each record to every other record in a given block with this approach, each block will have its own symmetric similarity matrix S . The corresponding distance matrix $D = 1 - S$ will be passed to an Agglomerative (hierarchical) Clustering model to assign each record a final Cluster ID. Agglomerative Clustering works by treating each record as its own cluster then iteratively merging the most similar clusters until a stopping criterion (`distance_threshold` hyperparameter) is reached or all records combine into a single cluster [4].

Field	Weight	Similarity Metric
BIRTHDATE	20%	Normalized Indel Similarity
FIRST_C	15%	Normalized Indel Similarity
FIRST_C_PHON	2.5%	Normalized Indel Similarity of Double Metaphone
LAST_C	15%	Normalized Indel Similarity
LAST_C_PHON	2.5%	Normalized Indel Similarity of Double Metaphone
ADDRESS_LINE_1	20%	Normalized Indel Similarity
ADDRESS_LINE_2	10%	Normalized Indel Similarity
ADDRESS_CITY	5%	Normalized Indel Similarity
ADDRESS_ZIP	5%	Normalized Indel Similarity
ADDRESS_GENDER	5%	Exact Match

3. Splink with Fellegi-Sunter Model

The final approach leverages Splink, a Python framework package designed for fast, accurate, and scalable probabilistic entity resolution [5]. The Splink package implements the Fellegi-Sunter model which utilizes discrete similarity levels (also called Comparison Levels) for each field unlike Method 2 which uses continuous similarity scores per-field. For example, when comparing two records based on their Street Address field, their similarity will fall into one of the following buckets:

- Exact Match
- Jaro-Winkler Similarity ≥ 0.9
- Jaro-Winkler Similarity ≥ 0.8
- Jaro-Winkler Similarity ≥ 0.7
- Jaro-Winkler Similarity < 0.7

The full list of discrete similarity levels is shown as SQL rules in Appendix A.2. Using discrete levels results in a different process for computing the overall similarity (a continuous value between 0 and 1) for two records. The high-level overview of this computation is as such:

Start with a prior, the probability that represents the belief that the two records being compared are a match *before* utilizing any record data. In this case, this is the probability that any two records which are drawn at random are a match. This value is referred to as λ and is the initial baseline value for the match probability between two records. For large datasets, λ is typically very small.

Next, compare the two records, and for each field being compared one-by-one add or subtract to the original match probability depending on what the discrete Comparison Level is between the

two fields (e.g Exact Match, JW-Similarity > 0.9, JW-Similarity > 0.8, etc.). The amount that gets added or subtracted are called *partial match weights* and are parameters that get estimated by the Fellegi-Sunter model. The sum of the prior (converted to match weight scale) and all partial match weights creates a final match weight, M_{obs} , between the two records

$$M_{prior} = \log\left(\frac{\lambda}{1 - \lambda}\right)$$

$$M_{obs} = M_{prior} + M_{fname} + M_{lname} + M_{street} + M_{city} + \dots$$

The final match weight, M_{obs} , can be converted into the final match probability, p_{obs} , using

$$p_{obs} = \frac{2^{M_{obs}}}{1 + 2^{M_{obs}}}$$

Similar to Method 1 and Method 2, p_{obs} represents the probability that the two records being compared represent the same individual and should be assigned the same Cluster ID. Lastly, rather than using a separate clustering model to assign the final Cluster IDs like Method 2, Splink represents the resulting similarity matrix for a given block as a graph (where nodes represent records and edges represent the similarity between two records) and uses a simple pruning threshold to where only edges at or above the threshold are considered valid. Any edges below the threshold get removed and each remaining connected sub-graph is considered a final clustering group.

Evaluation and Final Results

For Method 1, the precision, recall, and f1-score are plotted below at various thresholds. The threshold with the best f1-score (0.8829) was 58 which resulted in a precision and recall of 0.9309 and 0.8396 respectively.

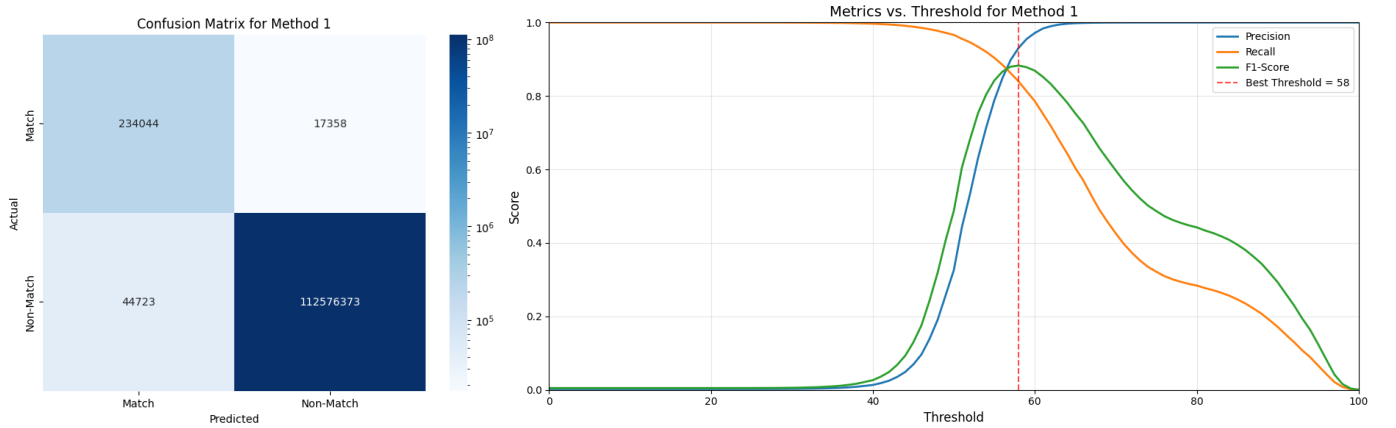


Figure 1: Metrics Plot at Various Threshold and Best Confusion Matrix for Method 1

For Method 2, the `distance_threshold` hyper-parameter (the linkage distance threshold where any value at or above will result in clusters not being merged) which resulted in the highest f1-score (0.8592) was 0.45. This resulted in a precision and recall of 0.9674 and 0.7728 respectively. The Confusion Matrix associated with this model is shown below:

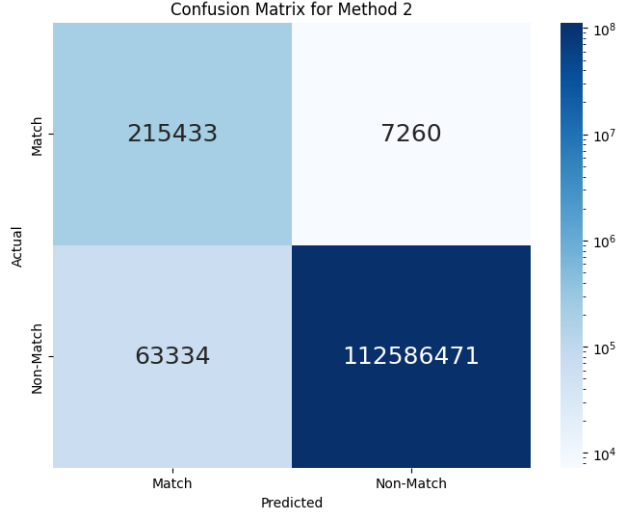


Figure 2: Best Confusion Matrix for Method 2

For Method 3, the model estimates for each discrete similarity level are shown below:

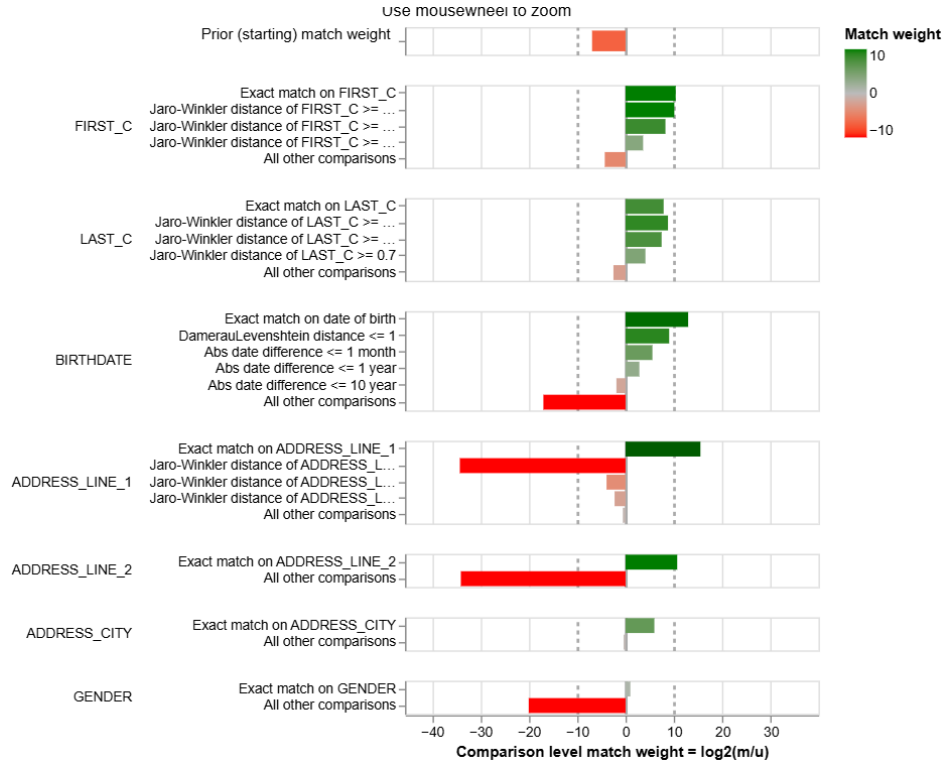


Figure 3: Comparison Level Match Weights for Splink Model

The more positive the comparison level match weight is the more likely the two records being compared are a match (and vice versa for negative values). For instance, if there is an exact match on ADDRESS_LINE_1 it is $2^{15.48} \approx 45639$ times more likely the records being compared are a match.

The pruning threshold where only edges at or above the threshold are considered valid links with

the best f1-score (0.965) was 0.867. This resulted in a precision and recall of 0.983 and 0.947 respectively. A screenshot of the interactive plot Splink provides to determine this value is shown below along with the corresponding Confusion Matrix:

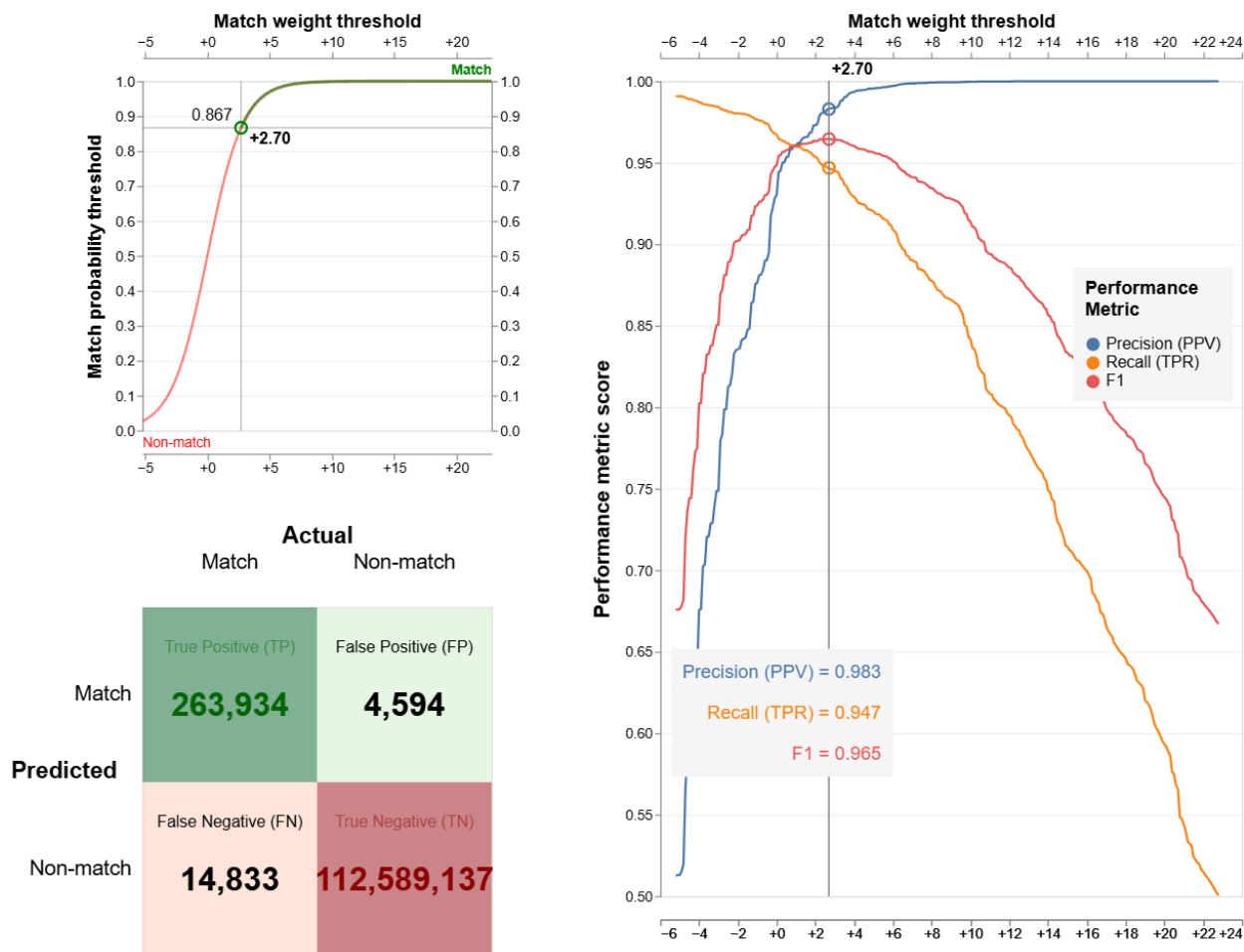


Figure 4: Splink Interactive Chart for Determining Best Threshold

Due to the sheer number of True Negatives in the context of this problem the accuracy for all three methods is approximately 100%, however the remaining results can be combined into a single table:

Method	Precision	Recall	f1-score
1. Simple String Comparison	0.9309	0.8396	0.8829
2. Custom Similarity Formula with Agglomerative Clustering	0.9674	0.7728	0.8592
3. Splink with Fellegi-Sunter Model	0.9830	0.947	0.9650

Based on these results it clear that the Splink model is the most robust and is best at clustering records regardless of data quality issues. Another interesting result is the Simple String Comparison Method resulted in a 3% higher f1-score than the Custom Similarity Formula with Agglomerative

Clustering model. A plausible reason for this is it may be possible that the set of fields/weights chosen in Method 2 are sub-optimal, as fine-tuning these values is a time-consuming process.

References

- [1] Centers for Medicare & Medicaid Services. *Reporting Shared MSIS Identification Numbers (Eligibility)*. <https://www.medicare.gov/tmsis/dataguide/t-msis-coding-blog/reporting-shared-msis-identification-numbers-eligibility/>. Accessed: 2025-10-12.
- [2] Walonoski, J., Kramer, M., Nichols, J., Quina, A., Moesel, C., Hall, D., Duffett, C., Dube, K., Gallagher, T., and McLachlan, S. (2018). Synthea: An approach, method, and software mechanism for generating synthetic patients and the synthetic electronic health care record. *Journal of the American Medical Informatics Association*, 25(3):230–238. 10.1093/jamia/ocx079.
- [3] Bachmann, M. (2025). *Levenshtein: Python extension for computing string edit distances and similarities*. <https://rapidfuzz.github.io/Levenshtein/levenshtein.html>. Accessed: 2025-11-29.
- [4] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, E. (2011–2025). 2.3.6. *Hierarchical clustering*. <https://scikit-learn.org/stable/modules/clustering.html#hierarchical-clustering>. Accessed: 2025-11-30.
- [5] Linacre, R., Lindsay, S., Manassis, T., Slade, Z., Hepworth, T., Kennedy, R., and Bond, A. (2022). Splink: Free software for probabilistic record linkage at scale. *International Journal of Population Data Science*, 7(3). 10.23889/ijpds.v7i3.1794.

Appendix

A.1 Procedure for Duplication

The following procedure was taken to duplicate and partially alter the 50,000 original records:

1. Take a random sample (without replacement), S , which is 80% of the original records.
2. For each record, $s \in S$, duplicate it $d \in [1, 5]$ times (d is chosen randomly for each s)
3. For each duplicated record, randomly select 3 to 5 fields from the list below to modify
 - FIRST_C, LAST_C, ADDRESS_LINE_1, BIRTHDATE, ADDRESS_CITY
4. For each selected field, modify its value from s using the table below

Field	Modification
FIRST_C	If selected, 15% chance to become missing value, otherwise 70% chance to swap to a similar first name and 30% chance to add a typo.
LAST_C	If selected, 10% chance to become missing value, otherwise 70% chance to swap to a similar last name and 30% chance to add a typo.
ADDRESS_LINE_1	If selected, 20% chance to become missing value, otherwise 70% chance to completely change street address, 20% chance to change just the suffix (e.g Street to St), and 10% to remain unchanged
BIRTHDATE	If selected, 5% chance to become missing value, otherwise 25% chance to swap the month/day, 25% chance to alter birth year +1/-1, 25% chance to alter the birth month by +1/-1, and 25% chance to alter birth day by +1/-1/+10/-10.
ADDRESS_CITY	If selected, 20% chance to become missing value, otherwise 90% chance to add a typo and 10% chance to remain unchanged

A.2 Full List of Discrete Similarity Levels for Splink

The list of all discrete similarity levels used in the Splink model is located below. They are defined using auto-generated SQL rules and each level corresponds to two parameters that must be estimated by the Fellegi-Sunter Model (called m -probabilities and u -probabilities). The Comparison Level Match Weights shown in Figure 3 correspond to $\log_2(\frac{m}{u})$ for the respective Comparison Level.

Comparison 'NameComparison' of "FIRST_C".

Similarity is assessed using the following ComparisonLevels:

- 'FIRST_C is NULL' with SQL rule: "FIRST_C_l" IS NULL OR "FIRST_C_r" IS NULL
- 'Exact match on FIRST_C' with SQL rule: "FIRST_C_l" = "FIRST_C_r"
- 'Jaro-Winkler distance of FIRST_C >= 0.92' with SQL rule:
 $\rightarrow$ jaro_winkler_similarity("FIRST_C_l", "FIRST_C_r") >= 0.92
- 'Jaro-Winkler distance of FIRST_C >= 0.88' with SQL rule:
 $\rightarrow$ jaro_winkler_similarity("FIRST_C_l", "FIRST_C_r") >= 0.88
- 'Jaro-Winkler distance of FIRST_C >= 0.7' with SQL rule:
 $\rightarrow$ jaro_winkler_similarity("FIRST_C_l", "FIRST_C_r") >= 0.7
- 'All other comparisons' with SQL rule: ELSE

Comparison 'NameComparison' of "LAST_C".

Similarity is assessed using the following ComparisonLevels:

- 'LAST_C is NULL' with SQL rule: "LAST_C_l" IS NULL OR "LAST_C_r" IS NULL
- 'Exact match on LAST_C' with SQL rule: "LAST_C_l" = "LAST_C_r"
- 'Jaro-Winkler distance of LAST_C >= 0.92' with SQL rule:
 $\rightarrow$ jaro_winkler_similarity("LAST_C_l", "LAST_C_r") >= 0.92
- 'Jaro-Winkler distance of LAST_C >= 0.88' with SQL rule:
 $\rightarrow$ jaro_winkler_similarity("LAST_C_l", "LAST_C_r") >= 0.88
- 'Jaro-Winkler distance of LAST_C >= 0.7' with SQL rule:
 $\rightarrow$ jaro_winkler_similarity("LAST_C_l", "LAST_C_r") >= 0.7
- 'All other comparisons' with SQL rule: ELSE

Comparison 'DateOfBirthComparison' of "BIRTHDATE".

Similarity is assessed using the following ComparisonLevels:

- 'transformed BIRTHDATE is NULL' with SQL rule: try_strptime("BIRTHDATE_l", '%Y-%m-%d') IS NULL OR try_strptime("BIRTHDATE_r", '%Y-%m-%d') IS NULL
- 'Exact match on date of birth' with SQL rule: "BIRTHDATE_l" = "BIRTHDATE_r"
- 'DamerauLevenshtein distance <= 1' with SQL rule:
 $\rightarrow$ damerau_levenshtein("BIRTHDATE_l", "BIRTHDATE_r") <= 1

- 'Abs date difference <= 1 month' with SQL rule:
 - ABS(EPOCH(try_strptime("BIRTHDATE_l", '%Y-%m-%d')) -
 - EPOCH(try_strptime("BIRTHDATE_r", '%Y-%m-%d')))) <= 2629800.0
- 'Abs date difference <= 1 year' with SQL rule:
 - ABS(EPOCH(try_strptime("BIRTHDATE_l", '%Y-%m-%d')) -
 - EPOCH(try_strptime("BIRTHDATE_r", '%Y-%m-%d')))) <= 31557600.0
- 'Abs date difference <= 10 year' with SQL rule:
 - ABS(EPOCH(try_strptime("BIRTHDATE_l", '%Y-%m-%d')) -
 - EPOCH(try_strptime("BIRTHDATE_r", '%Y-%m-%d')))) <= 315576000.0
- 'All other comparisons' with SQL rule: ELSE

Comparison 'JaroWinklerAtThresholds' of "ADDRESS_LINE_1".

Similarity is assessed using the following ComparisonLevels:

- 'ADDRESS_LINE_1 is NULL' with SQL rule: "ADDRESS_LINE_1_l" IS NULL OR
 - "ADDRESS_LINE_1_r" IS NULL
- 'Exact match on ADDRESS_LINE_1' with SQL rule: "ADDRESS_LINE_1_l" =
 - "ADDRESS_LINE_1_r"
- 'Jaro-Winkler distance of ADDRESS_LINE_1 >= 0.9' with SQL rule:
 - jaro_winkler_similarity("ADDRESS_LINE_1_l", "ADDRESS_LINE_1_r") >= 0.9
- 'Jaro-Winkler distance of ADDRESS_LINE_1 >= 0.8' with SQL rule:
 - jaro_winkler_similarity("ADDRESS_LINE_1_l", "ADDRESS_LINE_1_r") >= 0.8
- 'Jaro-Winkler distance of ADDRESS_LINE_1 >= 0.7' with SQL rule:
 - jaro_winkler_similarity("ADDRESS_LINE_1_l", "ADDRESS_LINE_1_r") >= 0.7
- 'All other comparisons' with SQL rule: ELSE

Comparison 'ExactMatch' of "ADDRESS_LINE_2".

Similarity is assessed using the following ComparisonLevels:

- 'ADDRESS_LINE_2 is NULL' with SQL rule: "ADDRESS_LINE_2_l" IS NULL OR
 - "ADDRESS_LINE_2_r" IS NULL
- 'Exact match on ADDRESS_LINE_2' with SQL rule: "ADDRESS_LINE_2_l" =
 - "ADDRESS_LINE_2_r"
- 'All other comparisons' with SQL rule: ELSE

Comparison 'ExactMatch' of "ADDRESS_CITY".

Similarity is assessed using the following ComparisonLevels:

- 'ADDRESS_CITY is NULL' with SQL rule: "ADDRESS_CITY_l" IS NULL OR "ADDRESS_CITY_r"
 - IS NULL
- 'Exact match on ADDRESS_CITY' with SQL rule: "ADDRESS_CITY_l" = "ADDRESS_CITY_r"
- 'All other comparisons' with SQL rule: ELSE

Comparison 'ExactMatch' of "GENDER".

Similarity is assessed using the following ComparisonLevels:

- 'GENDER is NULL' with SQL rule: "GENDER_l" IS NULL OR "GENDER_r" IS NULL
- 'Exact match on GENDER' with SQL rule: "GENDER_l" = "GENDER_r"
- 'All other comparisons' with SQL rule: ELSE